

M.Tech Project Report

**LASER — Layer-Aware Selective Erasure via Residual Editing
for Zero-Shot Unlearning in Vision Transformers**

A THESIS

SUBMITTED FOR THE DEGREE OF

Master of Technology

IN

Artificial Intelligence

by

YALLA MAHANTH



Computer Science and Automation

Indian Institute of Science

BANGALORE – 560 012

MAY 2026

© YALLA MAHANTH

MAY 2026

All rights reserved

Declaration of Originality

I, **Yalla Mahanth**, SR No. **04-04-03-10-51-24-1-24004**, declare that the thesis titled

**LASER — Layer-Aware Selective Erasure via Residual Editing for
Zero-Shot Unlearning in Vision Transformers**

represents original work carried out by me in the **Department of Computer Science and Automation, Indian Institute of Science**, during **2024–2026**.

I solemnly certify, on my signature below, that: (i) the data and results have not been manipulated; (ii) no intellectual property has been plagiarised and all contributions of others have been clearly cited; (iii) all collaborative research and discussions have been explicitly acknowledged; and (iv) I am aware that any false claim will result in severe disciplinary action and that the work may be screened for academic misconduct.

Date: _____

Student Signature: _____

In my capacity as supervisor of the above-mentioned work, I certify that the above statements are true to the best of my knowledge, and I have carried out due diligence to ensure the originality of the report.

Advisor 1

Prof. Y. Narahari
Game Theory Lab (GTL)
Dept. of Computer Science and Automation
Indian Institute of Science

Signature: _____

Date: _____

Advisor 2

Prof. R. Venkatesh Babu
Vision and AI Lab (VAL)
Dept. of Computational and Data Sciences
Indian Institute of Science

Signature: _____

Date: _____

Acknowledgements

I would like to express my sincere gratitude to my guide, **Prof. Y. Narahari**, Game Theory Lab (GTL), Department of Computer Science and Automation, Indian Institute of Science, for his guidance, support, and encouragement throughout this work.

I am deeply grateful to my co-guide, **Prof. R. Venkatesh Babu**, Vision and Artificial Intelligence Lab (VAL), Department of Computational and Data Sciences, Indian Institute of Science, with whom I had the opportunity to work closely on this project. His insights and feedback were invaluable in shaping this work.

I am also grateful to the **Indian Institute of Science** for providing the institutional infrastructure and high-performance computing resources that made this research possible.

I also thank the faculty members, lab members, friends, and peers who supported me during the course of this project. Finally, I am grateful to my family for their constant encouragement and support.

Publications based on this Thesis

1. This work has been submitted to NeurIPS 2026 and is currently under review.

Abstract

*Machine unlearning aims to remove specified knowledge from a trained model without retraining it from scratch, but existing methods often depend on retained data, forget samples, synthetic proxies, or costly re-optimization. We study zero-shot class unlearning for Vision Transformers, where neither original training data nor auxiliary samples are available, and introduce LASER (**L**ayer-**A**ware **S**elective **E**rasure via **R**esidual Editing), a post-hoc method that exploits a key empirical finding, class-specific memory in ViTs is concentrated in late transformer blocks. LASER uses this insight to edit only a small subset of late layers with lightweight low-rank residual updates, leaving the rest of the network untouched. On ImageNet with ViT-B/16, it drives forget-class accuracy to 0% while preserving top-1 accuracy within a negligible drop relative to the original model, and generalizes across multiple ViT and DeiT variants. LASER is substantially more efficient than full retraining, completing each deletion request in under three hours on a single GPU and achieving a median $136\times$ speedup over full retraining, with a range of $105\text{--}171\times$ across all evaluated classes.*

Contents

Declaration of Originality	i
Acknowledgements	ii
Publications based on this Thesis	iii
Abstract	iv
Keywords	xi
1 Introduction	1
1.1 Contributions	2
2 Related Work	3
2.1 Machine unlearning	3
2.2 Zero-shot and data-free unlearning	3
2.3 Vision Transformers, LoRA, and layerwise representations	4
2.4 Positioning	4
3 Background and Problem Formulation	5
3.1 Standard machine unlearning	5
3.2 Class-wise unlearning	6
3.3 Zero-shot class unlearning	6
3.4 Problem statement	7
3.5 Vision Transformer structure	8
3.6 Sparse late-layer edits and layer selection	9
3.7 Evaluation Criteria	9
4 Method	11
4.1 Problem setup	11
4.2 Layer-local residual edit	12
4.3 Zero-shot noise and objective	13
4.4 Scoring and composition	14

5	Experiments and Results	16
5.1	Setup	16
5.2	Baselines	16
5.3	Main Results	17
5.4	Privacy - Utility Tradeoff and Layer-Count Ablation	18
5.5	Cross-Architecture Scalability	18
6	Discussion and Limitations	21
6.1	Structural versus superficial forgetting	21
6.2	Limitations	21
7	Conclusion	23
7.1	Broader impact	23
A	ViT Architecture, Editing Theory, and Layer Search	25
A.1	Image Tokenisation and Positional Encoding	25
A.2	The Transformer Block: Two Residual Sub-Layers	26
A.2.1	Sub-layer 1: Multi-Head Self-Attention (MHSA).	26
A.2.2	Sub-layer 2: Position-wise MLP.	27
A.2.3	Full block expansion.	27
A.3	Compact Notation and Its Precise Meaning	27
A.4	Classification Head and Final Prediction	28
A.5	Why MLP Edits Provide Localised Control	29
A.6	Depth-Wise Semantic Progression and the Late-Layer Hypothesis	29
A.7	Combinatorial Layer Search and the Late-Layer Pool	30
B	Runtime Methodology and Speedup Calculations	32
B.1	Full-Retraining Reference Time	32
B.2	GPU Equivalence: NVIDIA V100 vs. RTX A6000	33
B.3	Per-Class LASER Timing and Speedup Table	34
C	Per-Class Results on ViT-B/16	36
D	MIA Attack Trends: Privacy Improvement with K	38
D.1	Layer-Count Effect on Membership Inference	38
D.2	MIA Under Varying Attack Strength	39
E	Layer Scoring, Representation Localisation, and Class Difficulty	40
E.1	Which Layers Encode Which Classes?	40
E.2	Layer Preference Across Classes	40
E.3	Class Difficulty as a Function of Representation Depth	41

F	Architecture-Specific Detailed Results	43
F.1	Full Results Across Six Architectures	43
F.2	Observations by Architecture Family	44
F.2.1	ViT family.	44
F.2.2	DeiT family.	44
F.2.3	ViT-Base vs. DeiT-Base.	44
F.3	Scaling Behaviour with Model Depth	45
F.4	Planned Evaluation: Swin Transformers and Beyond	45
G	Extended Limitations and Future Directions	46
G.1	Certified versus approximate unlearning.	46
G.2	Computational complexity of LASER.	46
G.3	Sequential and multi-class deletion.	49
	References	50

List of Tables

5.1	Approximate zero-shot class unlearning on ImageNet, ViT-B/16. LASER averaged over all evaluated forget classes (FA = 0% in every case); RA $\pm$ std captures class-to-class variation. Time: median full-pipeline wall-clock on RTX A6000. Bold : best data-free value. †: requires training data. Baselines: cited literature.	17
5.2	Layer-count ablation on ViT-B/16 (class 24). Per-layer oracle times; speedup is relative to $\approx$ 296 h full retrain.	19
5.3	Architecture scalability of LASER (class 970, best layer combo). All FA = 0%; no training data used.	19
B.1	Hardware specifications for the NVIDIA V100 SXM2 and RTX A6000. Sources: NVIDIA product datasheets [29, 30].	33
B.2	Per-class LASER runtime and speedup on ViT-B/16, ImageNet-1k (RTX A6000, $K = 4$ candidate layers). Retrain reference: 296 h = 17,760 min (see §B.1).	35
C.1	Per-class best-combo results on ViT-B/16, ImageNet-1k. Baseline RA = 80.20%. All results yield FA = 0%. Classes ordered by ascending K (edit budget required).	36
D.1	MIA AUC as a function of layer count K , three ViT-B/16 classes. FA = 0% in all cases. MIA = 0.50 is ideal (random-chance, indistinguishable from retrain oracle).	38
E.1	Top-scoring layer and minimum edit budget per class. Score is the normalised layer-scoring value (higher = more concentrated evidence in that block). Scores ≥ 0.80 permit single-layer erasure ($K = 1$). . . .	41
F.1	Full architecture scalability results (class 970, ImageNet-1k, best-combo configuration). Baseline RA varies by architecture. All FA = 0%; no training data accessed.	43

G.1	Number of combination-search evaluations Ω under different settings. $K_{\text{pool}} = 4$ is the top-pool size used in all experiments. Unrestricted search uses the full candidate pool $ \mathcal{B}_{\text{late}} $ without the top- K_{pool} selec- tion step. Full-depth worst case uses all L blocks.	48
-----	---	----

List of Figures

4.1	LASER workflow. A frozen pretrained ViT is decomposed into patch embeddings, transformer blocks, and a classifier head. LASER extracts one candidate block K , trains only a low-rank edit inside this block using optimized activation-level noise, and then replaces the edited block back into the full model. Frozen parts are marked by snowflakes and the optimized/editable parts by fire icons.	12
5.1	(a) Retain and forget accuracy vs. baselines. [†] requires training data. LASER (hatched bar, $\pm$ std) matches or exceeds data-dependent methods on RA with no training data. NegGrad leaves residual FA = 2.4%, while Zero-Shot leaves FA = 18%, All other methods achieved FA $\approx$ 0%. (b) Runtime on a log scale. LASER (median 2.18 h, $\approx$ 136 $\times$ speedup) is faster than every data-dependent baseline and eliminates the residual FA of the zero-shot method. Baseline values from cited papers.	18
5.2	(a) Full-pipeline wall-clock time per deletion request (RTX A6000, 4-candidate pool, $N = 20$ ViT-B/16 classes). LASER completes in 104–169 min, achieving speedups of 105 $\times$ –171 $\times$ over full retraining (median 136 $\times$). (b) Retain-accuracy drop and FA across forget classes (ViT-B/16, left cluster) and architectures (right); FA = 0% across all entries; MIA annotated inside bars.	20

Keywords

Machine Unlearning, Vision Transformers, Zero Shot Unlearning, Layer Aware Editing, Class-wise Erasure.

Chapter 1

Introduction

Regulations such as GDPR and CCPA grant individuals the right to request deletion of their personal data from trained models [26, 38]. Satisfying such requests in deployed systems is non-trivial, since a model that has memorized a data point may leak it via membership inference or model inversion attacks [14, 34], even after raw data is deleted. Machine unlearning [2] formalises the requirement that a trained model behave as if specified data never participated in training.

Existing unlearning methods face a fundamental tension between fidelity and practicality. Exact unlearning via full retraining [2] is prohibitively expensive for large models. Data-partitioning strategies [1] require non-standard training pipelines. Gradient-based approximate methods [17, 18] require the forget or retain set, assumptions that fail in post-deployment scenarios. More recent approaches use synthetic data generation [5] or teacher-student distillation [22], introducing additional cost and design complexity. Critically, none of these methods are designed for the now-dominant Vision Transformer (ViT) architecture [9], whose structural properties offer unexploited leverage for efficient forgetting.

A key empirical observation drives our method: class-relevant information in ViTs is non-uniformly distributed across depth. Prior analyses show that ViT representations become increasingly semantic with depth, with early layers capturing

low-level visual patterns such as colors and textures, and later layers encoding more object- and class-level concepts [8, 16, 33]. This motivates targeted late-layer edits to suppress a target class while leaving most of the network unchanged.

We introduce LASER, a post-hoc, zero-shot class unlearning method for pre-trained ViTs. Given a pretrained ViT and a class to forget, LASER *(i)* searches a small candidate pool restricted to the final 30% of transformer blocks, *(ii)* applies lightweight low-rank residual edits [21] to the selected subset via a score-guided search, and *(iii)* returns an unlearned model whose non-targeted weights are entirely unchanged.

1.1 Contributions

- We identify depth-wise class memory localisation in ViTs as the key structural property enabling selective post-hoc forgetting, and provide empirical evidence across architectures and classes.
- We propose LASER, a zero-shot, data-free unlearning method that operates via score-guided low-rank residual edits to a small candidate set of late transformer blocks, backed by theoretical locality bounds.
- We demonstrate that a single-layer edit suffices for complete class forgetting across diverse ViT/DeiT variants, while multi-layer combinations ($K \leq 3$) improve privacy robustness measured by membership inference AUC, and yield order-of-magnitude runtime reductions over retraining.

Chapter 2

Related Work

2.1 Machine unlearning

Cao and Yang [2] formalised unlearning via summation-form algorithms; Ginart et al. [17] established the first rigorous deletion guarantees. SISA [1] bounds re-training cost via data partitioning. Scrubbing [18] uses Newton steps derived from the Fisher information matrix but requires both forget and retain sets. SCRUB [22] frames unlearning as constrained optimisation over retain performance; NegGrad [28] uses gradient ascent on the forget set; Amnesiac [19] relies on stored gradient checkpoints; SalUn [12] selects parameters via gradient-based saliency. All data-dependent methods assume access to forget or retain data at unlearning time – a constraint that fails in post-deployment scenarios.

2.2 Zero-shot and data-free unlearning

Chundawat et al. [5] introduced zero-shot machine unlearning via error-minimising noise and gated knowledge transfer, but their approach requires a generator network trained offline. Foster et al. [13] (SSD) uses the Fisher matrix to dampen forget-relevant parameters but still requires both forget and retain data. Eldan and

Russinovich [11] demonstrated approximate concept erasure from LLMs without re-training, highlighting the broader relevance of post-hoc forgetting. LASER differs from all of the above, it requires no forget data, no retain data, no generator, and no offline preparation.

2.3 Vision Transformers, LoRA, and layerwise representations

ViTs [9] now dominate visual recognition, DeiT [36] and Swin [24] extend them with distillation-based and hierarchical designs. LoRA [21] shows that weight updates for adapting large pretrained transformers can be captured by low-rank matrix decompositions – a principle LASER applies in the unlearning direction. Prior analyses suggest that ViT representations evolve with depth, with early layers capture lower-level visual patterns, while later layers become increasingly semantic and object-centric [16, 33]. In parallel, Raghu et al. [33] show that self-attention enables early aggregation of global information and that residual connections strongly propagate lower-layer features to later blocks. These observations motivate focusing unlearning on later Transformer blocks, while treating the choice of edited layers as an empirical question.

2.4 Positioning

LASER uniquely combines three properties absent in any single prior method: *(i)* strict zero-shot operation requiring no original training, forget, or retain data, *(ii)* explicit targeting of the ViT residual architecture via score-guided late-layer selection, and *(iii)* class-level granularity with sub-percent utility loss. This combination makes LASER directly applicable to deployed ViT systems without any modification to the training or inference pipeline.

Chapter 3

Background and Problem Formulation

3.1 Standard machine unlearning

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ be a labeled dataset with inputs $x_i \in \mathcal{X}$ and labels $y_i \in [C] := \{0, \dots, C-1\}$. A classifier $f_\theta : \mathcal{X} \rightarrow \Delta^{C-1}$ is trained by empirical risk minimization,

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_\theta(x_i), y_i),$$

where ℓ is cross-entropy. Given a forget request $\mathcal{D}_f \subset \mathcal{D}$, the retain set is $\mathcal{D}_r = \mathcal{D} \setminus \mathcal{D}_f$. Machine unlearning seeks parameters θ' such that $f_{\theta'}$ behaves like a model retrained only on $\mathcal{D}_r$, i.e., $f_{\theta'} \approx f_{\theta_r^*}$, where

$$\theta_r^* = \arg \min_{\theta} \frac{1}{|\mathcal{D}_r|} \sum_{(x,y) \in \mathcal{D}_r} \ell(f_\theta(x), y).$$

Exact unlearning requires distributional equivalence to retraining [2, 17], but is usually as costly as retraining itself. Approximate methods relax this requirement to bounded divergence, yet commonly require access to $\mathcal{D}_f$, $\mathcal{D}_r$, or original-training

gradients [18, 22].

3.2 Class-wise unlearning

We study class-wise unlearning, where the forget request specifies a target class $c_f \in [C]$:

$$\mathcal{D}_f = \{(x, y) \in \mathcal{D} : y = c_f\}, \quad \mathcal{X}_f = \{x : (x, c_f) \in \mathcal{D}\}.$$

A successful unlearned model must: *forget* the target class by assigning negligible probability to c_f on $\mathcal{X}_f$; *preserve utility* by maintaining accuracy on classes $c \neq c_f$; *preserve calibration* by keeping retained-class logits close to the original model; and *reduce privacy leakage* so that membership inference attacks on $\mathcal{D}_f$ are near random guessing, with AUC close to 0.5 [34].

3.3 Zero-shot class unlearning

Definition 1 (Zero-Shot Class Unlearning). *Given a pretrained model f_θ and a target class $c_f \in [C]$, a zero-shot class-unlearning operator U_{zs} produces*

$$\theta' = U_{\text{zs}}(\theta, c_f),$$

without access to the original training set $\mathcal{D}$, forget set $\mathcal{D}_f$, retain set $\mathcal{D}_r$, optimizer state, or gradients from the original training procedure.

This setting is stricter than data-dependent unlearning and is motivated by deployment, training data may already have been deleted for legal or privacy reasons [26, 38], or may never be available to the unlearning service. Requiring such data during deletion would weaken the privacy guarantee. Prior zero-shot methods may still rely on a pretrained generator, which acts as an indirect data source [5]. LASER removes this dependency. The only auxiliary information LASER may use

is a small set of public, non-sensitive probe inputs $\mathcal{P}$, sampled independently of $\mathcal{D}$, used only to score candidate edits and never to reconstruct or approximate $\mathcal{D}_f$. This probe set is not mandatory in the strictest zero-shot setting, candidate edits can instead be ranked using only the model’s own logits and confidence scores, without any external inputs. Thus, $\mathcal{P}$ provides a practical trade-off, it can make edit selection more stable, while omitting it preserves a purely zero-shot unlearning protocol.

3.4 Problem statement

Given a pretrained Vision Transformer f_θ , a target class c_f , and no access to training data, the goal is to construct an edited model $f_{\theta'}$ by modifying only a small set of transformer blocks $\mathcal{S}$:

$$\theta' = \theta + \Delta\theta_{\mathcal{S}}, \quad \Delta\theta_{\mathcal{S}} = \{\Delta\theta_b : b \in \mathcal{S}\}, \quad \Delta\theta_b = 0 \quad \forall b \notin \mathcal{S}, \quad |\mathcal{S}| \leq K_{\text{edit}}.$$

where $b \in \{1, \dots, L\}$ indexes transformer blocks and K_{edit} is the maximum number of blocks allowed to be edited. The edit is chosen to suppress the target class while preserving retained behavior by

$$\min_{\mathcal{S}, \delta\theta_{\mathcal{S}}} \lambda_f \mathcal{L}_{\text{forget}} + \lambda_u \mathcal{L}_{\text{utility}} + \lambda_r \mathcal{R}(\delta\theta_{\mathcal{S}}). \quad (3.1)$$

Let $z_b(n)$ denote the pre-softmax logits obtained by passing a synthetic activation n through edited block b and the frozen classifier head. The forget loss is

$$\mathcal{L}_{\text{forget}} = z_{b,c_f}(n_{c_f}) - \max_{j \neq c_f} z_{b,j}(n_{c_f}), \quad (3.2)$$

and the utility loss preserves non-target synthetic activations as

$$\mathcal{L}_{\text{utility}} = \mathcal{L}_{\text{CE}}(z_b(n_{-c_f}), y_{-c_f}). \quad (3.3)$$

subject to the zero-shot constraints above. Here $\mathcal{L}_{\text{forget}}$ suppresses class- c_f evidence, $\mathcal{L}_{\text{utility}}$ penalizes retained-class drift from the original model, and $\mathcal{R}$ controls the magnitude of the selected block edits. In LASER, each block update $\Delta\theta_b$ for $b \in \mathcal{S}$ is implemented later as a low-rank LoRA perturbation, so the pretrained weights remain frozen and only a small number of adapter parameters are optimized [21].

3.5 Vision Transformer structure

A Vision Transformer partitions an image into M non-overlapping patches, linearly projects each to a d -dimensional token, prepends a learnable [CLS] token, adds positional embeddings, and feeds the resulting sequence $h_0 \in \mathbb{R}^{(M+1) \times d}$ through L stacked transformer blocks [9]. Each block l comprises two *independent* residual sub-layers, multi-head self-attention (MHSA) and a position-wise MLP, each with its own skip connection and layer normalisation [37, 39]:

$$a_l = h_{l-1} + \text{MHSA}(\text{LN}(h_{l-1})), \quad (3.4)$$

$$h_l = a_l + \text{MLP}(\text{LN}(a_l)). \quad (3.5)$$

For compact notation, we write each block as $h_l = h_{l-1} + F_l(h_{l-1})$, where F_l denotes the combined non-identity contribution of Equations (3.4) and (3.5); the precise expansion and its implications for localised editing are given in Chapter A. The final classifier applies a linear head g to the [CLS] token of the last block:

$$f_\theta(x) = \text{softmax}(g(h_L^{[\text{CLS}]})). \quad (3.6)$$

DeiT uses the same encoder equations with a distillation token, Swin replaces global attention with shifted-window attention [24, 36]. The two-level residual structure in Equations (3.4) and (3.5) makes localised editing natural, a change confined to MLP_l alters only the additive residual $h_l - a_l$ while leaving a_l , all earlier blocks, and

the identity path entirely intact.

3.6 Sparse late-layer edits and layer selection

ViTs exhibit depth-wise semantic progression, early blocks encode broad, class-agnostic features while later blocks concentrate task-discriminative evidence for specific classes [9, 33], so targeting only late layers is both principled and sufficient. The two-level residual structure further ensures that a low-norm edit at block ℓ induces a controlled shift in the final representation, bounded by the Lipschitz constant of the unedited suffix (see Section A.6). Because class- c_f logit evidence decomposes approximately additively across blocks, suppressing only those blocks in the late-layer set $\mathcal{S}$ collapses the target-class score while leaving retained classes intact. Selecting $\mathcal{S}$ exactly is combinatorial. LASER restricts candidates to the final $\alpha \approx 0.30$ fraction of blocks and evaluates at most 14 combinations per forget class, reducing search cost by a factor of α^K relative to the full subset search (see Section A.7).

3.7 Evaluation Criteria

We evaluate unlearning along four axes: forgetting, utility, privacy, and efficiency. Forgetting is measured by *Forget Accuracy* (FA), the top-1 accuracy of the edited model $f_{\theta'}$ on the target-class inputs $\mathcal{X}_f$, lower is better, with the ideal value being $\text{FA} = 0\%$. Utility is measured by *Retain Accuracy* (RA), the top-1 accuracy on all non-target classes $[C] \setminus \{c_f\}$, together with the retain accuracy drop $\Delta\text{RA} = \text{RA}(f_{\theta^*}) - \text{RA}(f_{\theta'})$ in percentage points. Privacy is measured using MIA AUC under a standard membership inference attack, which tests whether an example was part of the training set [34], an AUC near 0.5 corresponds to random guessing, while larger values indicate stronger residual membership leakage. Finally, efficiency is measured by wall-clock runtime on a single NVIDIA RTX A6000 GPU and by the fraction of model parameters modified during unlearning, the runtime protocol and

speedup calculations are detailed in Chapter B.

Chapter 4

Method

4.1 Problem setup

Let f_θ be a pretrained ViT with patch embedding E , transformer blocks $F_0, \dots, F_{L-1}$, and classifier head g . We use the compact block-level notation

$$h_0 = E(x), \quad h_{b+1} = h_b + F_b(h_b), \quad b = 0, \dots, L-1,$$

where F_b denotes the combined non-identity computation inside block b , the attention/MLP residual expansion is given in Chapter A. The classifier predicts

$$f_\theta(x) = \text{softmax}(g(h_L^{[\text{CLS}]})). \quad (4.1)$$

Given a target forget class c_f , LASER constructs an edited model $f_{\theta'}$ that removes evidence for c_f while preserving retain-class behavior. The method is zero-shot with respect to the original training data, instead of replaying training samples, it uses the frozen model, synthetic hidden-state noise, and a small probe set only for scoring.

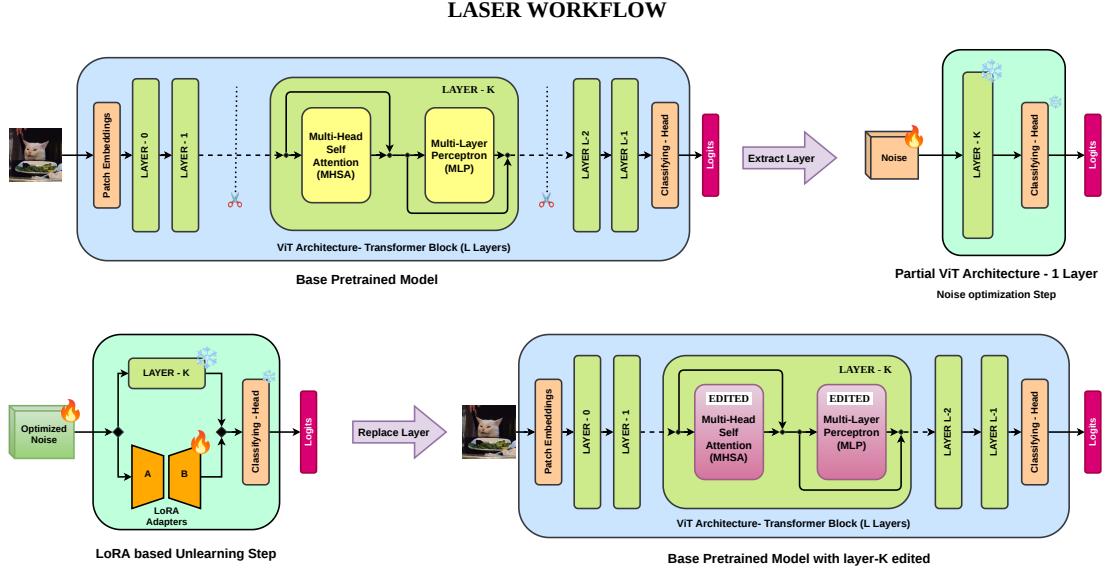


Figure 4.1: **LASER workflow.** A frozen pretrained ViT is decomposed into patch embeddings, transformer blocks, and a classifier head. LASER extracts one candidate block K , trains only a low-rank edit inside this block using optimized activation-level noise, and then replaces the edited block back into the full model. Frozen parts are marked by snowflakes and the optimized/editable parts by fire icons.

4.2 Layer-local residual edit

The workflow in Figure 4.1 edits one candidate block at a time. LASER restricts the search to the late-block pool

$$\mathcal{B}_{\text{late}} = \{b \in \{0, \dots, L-1\} : b \geq \lfloor (1-\rho)L \rfloor\}, \quad (4.2)$$

where $\rho \in (0, 1]$ keeps the final fraction of blocks. For each $b \in \mathcal{B}_{\text{late}}$, LASER copies block F_b , freezes all pretrained weights, and trains only low-rank LoRA updates inside the block.

For an edited weight matrix $W_0^{(b,m)}$ in module m of block b , LASER uses

$$W^{(b,m)} = W_0^{(b,m)} + \Delta W^{(b,m)}, \quad \Delta W^{(b,m)} = \frac{\alpha_{\text{LoRA}}}{r} B^{(b,m)} A^{(b,m)}, \quad \text{rank}(\Delta W^{(b,m)}) \leq r, \quad (4.3)$$

where $W_0^{(b,m)}$ is frozen, $A^{(b,m)}$ and $B^{(b,m)}$ are trainable LoRA matrices, $r \ll d$ is the LoRA rank, and α_{LoRA} is the LoRA scaling factor [21]. After training, the edited block is inserted back at the same index b . Because Equation (4.1) is residual, the edit changes only the learned residual branch of block b , while the skip path, earlier blocks, later frozen blocks, and classifier head remain fixed.

4.3 Zero-shot noise and objective

For a candidate block b , LASER optimizes a noise bank $N_b = \{n_c^{(b)}\}_{c=1}^C$, where each $n_c^{(b)}$ has the same shape as the input activation to block b . This noise is not an image, it is a synthetic hidden-state probe. First, N_b is optimized through the frozen block-head tail so that each noise vector elicits class-specific logits:

$$N_b^* = \arg \min_{N_b} \frac{1}{C} \sum_{c=1}^C \mathcal{L}_{\text{CE}} \left(g \left(F_b(n_c^{(b)})^{[\text{CLS}]} \right), c \right). \quad (4.4)$$

Then the LoRA adapter in block b is trained with a forget-preserve objective. Let

$$z_b(n) = g \left(F'_b(n)^{[\text{CLS}]} \right)$$

denote the pre-softmax logits produced by the edited block F'_b and frozen classifier head. The adapter objective is

$$\min_{\{\Delta W^{(b,m)}\}} \lambda_f \left[z_{b,c_f}(n_{c_f}^{(b)}) - \max_{j \neq c_f} z_{b,j}(n_{c_f}^{(b)}) \right] + \lambda_r \mathcal{L}_{\text{CE}} \left(z_b(n_{c_f}^{(b)}), y_{c_f} \right) + \lambda_w \sum_m \left\| \Delta W^{(b,m)} \right\|_F^2. \quad (4.5)$$

The first term lowers target-class evidence, the second preserves non-target class behavior, and the third keeps the LoRA edit small. Optionally, the retain term can

be supplemented by KL matching to the frozen model logits.

4.4 Scoring and composition

Each trained candidate block yields one edited model. LASER scores candidate block b using forget accuracy (FA), retain accuracy (RA), and membership-inference leakage (MIA):

$$S(b) = \alpha_s(\text{FA}_0 - \text{FA}_b) - \beta_s(\text{RA}_0 - \text{RA}_b) - \gamma_s|\text{MIA}_b - 0.5|, \quad (4.6)$$

where FA_0, RA_0 are baseline forget/retain accuracies, FA_b, RA_b are post-edit values for block b , and $\text{MIA}_b \approx 0.5$ means the forget examples are hard to distinguish by a membership-inference attack. The best single edit is

$$b^* = \arg \max_{b \in \mathcal{B}_{\text{late}}} S(b).$$

If stronger forgetting is required, LASER evaluates small subsets of top-ranked adapters and returns the best subset

$$\mathcal{S}^* \subseteq \text{Top}_{K_{\text{pool}}}(\mathcal{B}_{\text{late}}; S), \quad |\mathcal{S}^*| \leq K_{\text{edit}}, \quad (4.7)$$

with the final edited parameters obtained by applying the selected block-local LoRA updates:

$$\theta' = \theta + \sum_{b \in \mathcal{S}^*} \Delta\theta_b.$$

Algorithm 1 LASER: Layer-Aware Selective Erasure via Residual Editing

Require: Frozen ViT f_θ with blocks $F_0, \dots, F_{L-1}$ and head g ; forget class c_f ; candidate fraction ρ ; LoRA rank r ; top-pool size K_{pool} ; edit budget K_{edit} ; optional probe set $\mathcal{P}$.

Ensure: Edited model $f_{\theta'}$ with $\text{FA}(c_f) \approx 0\%$.

- 1: $\mathcal{B}_{\text{late}} \leftarrow \{b \in \{0, \dots, L-1\} : b \geq \lfloor (1 - \rho)L \rfloor\}$ ▷ Equation (4.2)
 - 2: **for** $b \in \mathcal{B}_{\text{late}}$ **do**
 - 3: $N_b^* \leftarrow \arg \min_{N_b} \frac{1}{C} \sum_{c=1}^C \mathcal{L}_{\text{CE}}(g((n_c^{(b)} + F_b(n_c^{(b)}))^{\text{[CLS]}}), c)$ ▷ Equation (4.4)
 - 4: **for** edited modules m in block b **do** ▷ Equation (4.3)
 - 5: $U^{(b,m)} \leftarrow B^{(b,m)} A^{(b,m)}, \Delta W^{(b,m)} \leftarrow \frac{\alpha_{\text{LoRA}}}{r} U^{(b,m)}$
 - 6: $W^{(b,m)} \leftarrow W_0^{(b,m)} + \Delta W^{(b,m)}$
 - 7: **end for** ▷ Equation (4.3)
 - 8: $z_b(n) = g((n + F'_b(n))^{\text{[CLS]}})$.
 - 9: Optimize all LoRA parameters in block b : ▷ Equation (3.1)
 - 10: $\min_{\{A^{(b,m)}, B^{(b,m)}\}_m} \lambda_f \mathcal{L}_{\text{forget}} + \lambda_u \mathcal{L}_{\text{utility}} + \lambda_r \mathcal{R}$ ▷ Equation (4.6)
 - 11: $S(b) \leftarrow \alpha_s(\text{FA}_0 - \text{FA}_b) - \beta_s(\text{RA}_0 - \text{RA}_b) - \gamma_s |\text{MIA}_b - 0.5|$
 - 12: **end for**
 - 13: $\mathcal{S}^* \leftarrow \arg \max_{\mathcal{S} \subseteq \mathcal{B}_{\text{top}}, |\mathcal{S}| \leq K_{\text{edit}}} S_{\text{combo}}(\mathcal{S})$ ▷ Equation (4.7)
 - 14: $\theta' \leftarrow \theta + \sum_{b \in \mathcal{S}^*} \Delta \theta_b$
 - 15: **return** $f_{\theta'}$
-

Chapter 5

Experiments and Results

5.1 Setup

We evaluate on ImageNet [7] (1,000 classes) using publicly available pretrained checkpoints for ViT-Small/Base/Large-16 [9] and DeiT-Tiny/Small/Base [36]. One class per trial is designated the *forget class*; no training, retain, or forget-class data is accessed at any stage of LASER (Algorithm 1). All experiments run on a single NVIDIA RTX A6000 (48 GB). FA, RA, and MIA are as defined in §3; no certified deletion is claimed.

5.2 Baselines

We compare against full *Retrain* from scratch, using the single-GPU equivalent of the $37\text{ h} \times 8\text{ V100}$ DeiT-B figure from Touvron et al. [36], giving a conservative reference of ≈ 296 GPU-hours on one device (V100 A6000 equivalence in Appendix B). Data-dependent baselines: *NegGrad* [28], *Amnesiac* [19], *Bad Teacher* [4], *SCRUB* [22], and *SalUn* [12], all require the full retain set. The data-free zero-shot method of Chundawat et al. [5] applies a fixed suppression strategy with no layer selection or adapter training.

5.3 Main Results

LASER achieves complete erasure ($\text{FA} = 0\%$) across all evaluated classes while preserving an average retain accuracy of **80.01%**, a drop of only 0.19 percentage points (pp) from the pretrained baseline of 80.20%, with a mean MIA of 0.30. Per-class RA drop ranges from 0.04 pp to 0.60 pp and MIA from 0.10 to 0.54, reflecting variation in how class evidence is distributed across layers (Appendix C). The full candidate-layer pipeline completes in a median of **130.7 minutes** per deletion request, a **136** $\times$ speedup over single-GPU full retraining, with a range of $105\times\text{--}171\times$ (Appendix B). Table 5.1 and Figure 5.1 compare LASER against all baselines on accuracy and runtime.

Relative to data-free methods, LASER eliminates the 18% residual FA of Chundawat et al. [5] at the cost of a modest 1.88 h additional runtime. Relative to data-dependent methods, it matches or exceeds retain accuracy without accessing any training data and is $60\times\text{--}140\times$ faster than each of them.

Table 5.1: Approximate zero-shot class unlearning on ImageNet, ViT-B/16. LASER averaged over all evaluated forget classes ($\text{FA} = 0\%$ in every case); $\text{RA} \pm \text{std}$ captures class-to-class variation. Time: median full-pipeline wall-clock on RTX A6000. **Bold**: best data-free value. †: requires training data. Baselines: cited literature.

Method	Data?	FA % $\downarrow$	RA % $\uparrow$	MIA $\rightarrow$ 0.5	Time
Retrain [†]	✓	0.0	80.20	0.50	≈ 296 h
NegGrad [†]	✓	2.4	78.91	0.39	12.6 h
Amnesiac [†]	✓	0.0	79.83	0.44	8.3 h
Bad Teacher [†]	✓	0.0	79.54	0.41	9.1 h
SCRUB [†]	✓	0.0	80.11	0.37	14.8 h
SalUn [†]	✓	0.0	79.97	0.35	11.4 h
Zero-Shot	✗	18.0	79.22	0.46	0.3 h
LASER (ours)	✗	0.0	80.01 $\pm$ 0.19	0.30	2.18 h

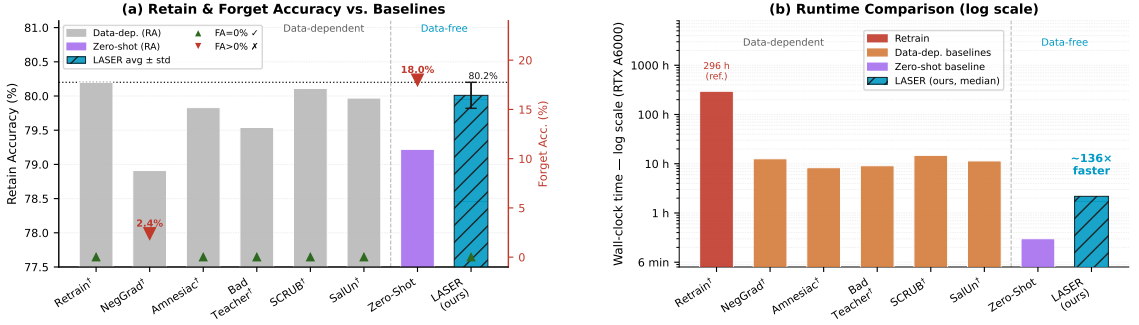


Figure 5.1: **(a)** Retain and forget accuracy vs. baselines. [†] requires training data. LASER (hatched bar, $\pm$ std) matches or exceeds data-dependent methods on RA with no training data. NegGrad leaves residual FA = 2.4%, while Zero-Shot leaves FA = 18%. All other methods achieved FA $\approx$ 0%. **(b)** Runtime on a log scale. LASER (median 2.18 h, $\approx$ 136 $\times$ speedup) is faster than every data-dependent baseline and eliminates the residual FA of the zero-shot method. Baseline values from cited papers.

5.4 Privacy - Utility Tradeoff and Layer-Count Ablation

Table 5.2 sweeps K on ViT-B/16, class 24, using per-layer oracle times. A single edited layer ($K = 1$, layer L11) drives FA to 0% with only a 0.04 pp RA drop in 32 min. Each additional layer progressively improves MIA, $K = 2$ reaches MIA = 0.16, and $K = 3$ achieves MIA = 0.21, moving toward random-chance ($= 0.5$) while retaining competitive RA. This trend confirms that composing additional adapters deepens privacy protection at a modest utility and runtime cost, detailed MIA attack trends and per-class breakdowns appear in Appendix D.

5.5 Cross-Architecture Scalability

Table 5.3 evaluates LASER across six ViT/DeiT architectures on a common forget class. FA = 0% is obtained on every model, with RA drops ranging from <0.01 pp (DeiT-Small) to 0.70 pp (ViT-Base), well within the accepted 5 pp utility threshold.

Table 5.2: Layer-count ablation on ViT-B/16 (class 24). Per-layer oracle times; speedup is relative to ≈ 296 h full retrain.

K	Layers	FA %	RA %	MIA	Speedup
0	None (baseline)	98.0	80.18	~ 0.50	—
1	{L11}	0.0	80.16	0.10	$560\times$
2	{L11, L9}	0.0	80.00	0.16	$280\times$
3	{L11, L9, L10}	0.0	79.85	0.21	$187\times$
—	Retrain (oracle)	0.0	80.20	0.50	$1\times$

The same late-layer search window ($\alpha = 0.30$) and LoRA rank transfer directly across model families and depths of 12–24 blocks without any architecture-specific tuning.

Table 5.3: Architecture scalability of LASER (class 970, best layer combo). All FA = 0%; no training data used.

Model	Params	FA %	RA %	Δ RA
DeiT-Small	22 M	0.0	78.84	−0.002
ViT-Large	307 M	0.0	83.05	−0.190
DeiT-Base	86 M	0.0	80.71	−0.110
DeiT-Tiny	6 M	0.0	69.07	−0.460
ViT-Small	22 M	0.0	71.15	−0.660
ViT-Base	86 M	0.0	79.48	−0.700

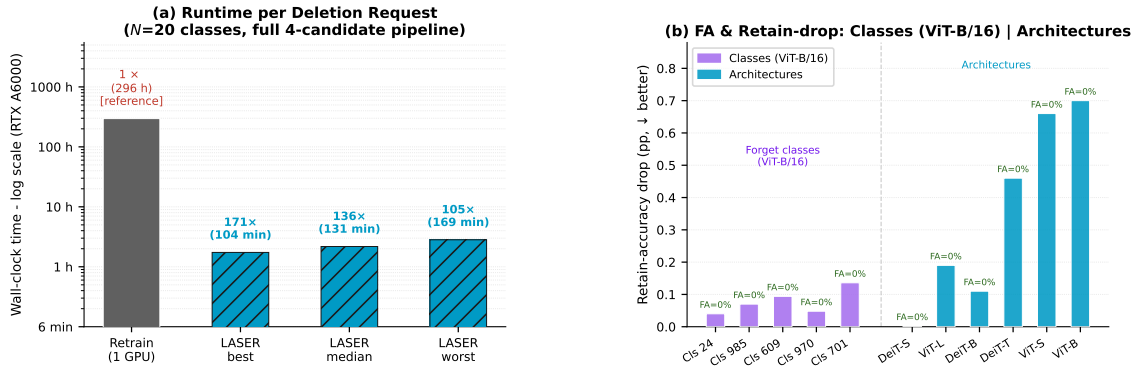


Figure 5.2: **(a)** Full-pipeline wall-clock time per deletion request (RTX A6000, 4-candidate pool, $N = 20$ ViT-B/16 classes). LASER completes in 104–169 min, achieving speedups of $105\times$ – $171\times$ over full retraining (median $136\times$). **(b)** Retain-accuracy drop and FA across forget classes (ViT-B/16, left cluster) and architectures (right); FA = 0% across all entries; MIA annotated inside bars.

Chapter 6

Discussion and Limitations

6.1 Structural versus superficial forgetting

The consistent reduction in MIA AUC from ≈ 0.50 pre-edit to values between 0.10 and 0.54 across all evaluated classes indicates that LASER modifies late-layer representations structurally, rather than merely suppressing the target logit at the output head [3]. An edit confined to the classification head would leave the internal activation distribution intact, providing no signal to a membership-inference attacker probing the full softmax vector, the consistent MIA reduction argues against this. As K increases from 1 to 3, MIA AUC continues to move toward random-chance (0.5), indicating that each additional layer progressively dissolves residual membership signal. The detailed per-layer and per-class MIA trends are presented in Appendix D.

6.2 Limitations

LASER does not provide certified deletion [20]; MIA AUC near 0.5 is a necessary but not sufficient condition for provable membership indistinguishability, and practitioners requiring formal guarantees should treat the method as a computationally

efficient complement to certified frameworks. The computational complexity of the combination-search phase grows with K and the candidate pool size; while the late-layer restriction ($\alpha = 0.30$) limits evaluations to at most 14 combinations for ViT-Base, larger candidate budgets or deeper pools could increase search time materially. Finally, the evaluation covers ViT and DeiT families on ImageNet; extension to Swin Transformers [24], vision-language models, and generative architectures, as well as to instance-level deletion and sequential multi-class schedules, is left to future work (Appendix G).

Chapter 7

Conclusion

We presented LASER, a post-hoc, data-free method for approximate zero-shot class unlearning in Vision Transformers. Exploiting the empirical finding that class-discriminative evidence concentrates in late transformer blocks, LASER scores candidate layers, trains lightweight low-rank adapters, and selects the optimal edit combination, entirely without access to training, forget, or retain data. Across all evaluated forget classes on ImageNet, the method drives forget-class accuracy to 0% while preserving an average retain accuracy of 80.01 % (± 0.19 pp), achieving a mean MIA AUC of 0.30 with a full pipeline runtime of 130.7 min per request (median $136\times$ speedup, range $105\times$ – $171\times$ over single-GPU retraining). The method generalises without modification across six ViT/DeiT architectures spanning 6 M to 307 M parameters, with RA drops of <0.01 pp to 0.70 pp.

7.1 Broader impact

LASER substantially lowers the computational and data-access barriers to GDPR- and CCPA-compliant right-to-erasure, reducing deletion time from days to hours on commodity hardware and removing the requirement to retain the original training

set. The layer-scoring procedure also serves as a diagnostic tool for localising class-specific representations within a pretrained ViT, a byproduct independently useful for model interpretability and targeted fine-tuning. The same editing capability could, however, be misused to silently suppress safety-critical or demographically significant classes; deployments should be accompanied by transparent audit logs and independent MIA verification. Reduced MIA AUC does not constitute a formal privacy guarantee: we recommend institutional governance review for deletion requests that could materially affect model behaviour at scale, and we release all code and configurations to support reproducibility and future research in practical machine unlearning.

Appendix A

ViT Architecture, Editing Theory, and Layer Search

LASER relies on two claims that warrant fuller treatment, (1) that the Vision Transformer’s residual structure makes block-level editing both localised and controlled, (2) that restricting the layer-selection search to a late-layer candidate pool is sufficient without sacrificing coverage. This appendix establishes both rigorously. §A.1–A.3 derive the complete encoder equations from image tokenisation through to the classification head, following the Pre-LN convention adopted by ViT and DeiT [9, 36, 39], in which layer normalisation is applied before each sub-layer. §A.6 formalises why low-norm MLP edits at late blocks produce controlled representation shifts. §A.7 analyses the combinatorial search problem and derives the cost reduction achieved by the late-layer pool.

A.1 Image Tokenisation and Positional Encoding

Given an image $x \in \mathbb{R}^{H \times W \times C}$, ViT divides it into $M = HW/P^2$ non-overlapping patches of size $P \times P$ pixels. Each patch $p_i \in \mathbb{R}^{P^2 C}$ is flattened and projected to a

d -dimensional embedding via a learnable linear map $E \in \mathbb{R}^{(P^2 C) \times d}$:

$$z_i = p_i E, \quad i = 1, \dots, M.$$

A learnable [CLS] token $z_0 \in \mathbb{R}^d$ is prepended to the sequence. Learnable positional embeddings $\Pi = [\pi_0, \pi_1, \dots, \pi_M] \in \mathbb{R}^{(M+1) \times d}$ are added element-wise:

$$h_0 = [z_0; z_1; \dots; z_M] + \Pi \in \mathbb{R}^{(M+1) \times d}.$$

This sequence h_0 is the input to the first transformer block.

A.2 The Transformer Block: Two Residual Sub-Layers

A transformer encoder layer consists of two distinct sub-operations, each wrapped in its own residual (skip) connection and layer normalisation. This is the key structural point, the single-block shorthand $h_l = h_{l-1} + F_l(h_{l-1})$ compresses two independent residual additions, not one.

A.2.1 Sub-layer 1: Multi-Head Self-Attention (MHSA).

Vaswani et al. [37] define multi-head attention as h parallel scaled dot-product attention heads whose outputs are concatenated and linearly projected. For a sequence $X \in \mathbb{R}^{N \times d}$ and head k :

$$\text{Attn}_k(X) = \text{softmax}\left(\frac{XW_k^Q(XW_k^K)^\top}{\sqrt{d_k}}\right)XW_k^V,$$

where $W_k^Q, W_k^K, W_k^V \in \mathbb{R}^{d \times d_k}$ with $d_k = d/h$. The multi-head output is

$$\text{MHSA}(X) = [\text{Attn}_1(X); \dots; \text{Attn}_h(X)]W^O, \quad W^O \in \mathbb{R}^{d \times d}.$$

Applied with a Pre-LN residual connection at block l :

$$a_l = h_{l-1} + \text{MHSA}\left(\text{LN}(h_{l-1})\right). \quad (\text{A.1})$$

The skip connection h_{l-1} passes through unmodified; only the attention branch is normalised before computation.

A.2.2 Sub-layer 2: Position-wise MLP.

The MLP sub-layer applies a two-layer feed-forward network independently to each token position:

$$\text{MLP}(u) = \sigma(uW_1 + b_1)W_2 + b_2,$$

where $W_1 \in \mathbb{R}^{d \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d}$, and σ is an activation (GELU in ViT/DeiT [9]).

The hidden dimension is typically $d_{\text{ff}} = 4d$. Applied with its own Pre-LN residual connection, and taking the attention output a_l , *not* h_{l-1} , as input:

$$h_l = a_l + \text{MLP}\left(\text{LN}(a_l)\right). \quad (\text{A.2})$$

A.2.3 Full block expansion.

Substituting Equation (A.1) into Equation (A.2) gives the complete single-block transformation:

$$h_l = h_{l-1} + \underbrace{\text{MHSA}\left(\text{LN}(h_{l-1})\right)}_{\text{attention residual}} + \text{MLP}\left(\text{LN}\left(h_{l-1} + \text{MHSA}\left(\text{LN}(h_{l-1})\right)\right)\right). \quad (\text{A.3})$$

A.3 Compact Notation and Its Precise Meaning

In the main text we write block l as

$$h_l = h_{l-1} + F_l(h_{l-1}),$$

where F_l denotes the combined non-identity contribution of the block. Comparing with Equation (A.3), this implicitly defines:

$$\begin{aligned} F_l(h_{l-1}) &= \text{MHSA}\left(\text{LN}(h_{l-1})\right) \\ &\quad + \text{MLP}\left(\text{LN}\left(h_{l-1} + \text{MHSA}\left(\text{LN}(h_{l-1})\right)\right)\right). \end{aligned} \quad (\text{A.4})$$

Three points about this definition are important:

1. F_l is *not* a simple sum $F_l^{\text{attn}} + F_l^{\text{mlp}}$ evaluated on the same input, because the MLP sub-layer receives the attention-updated intermediate a_l , not h_{l-1} .
2. The compact notation preserves the overall block structure $h_l = h_{l-1} + (\text{non-identity contribution})$ and is the standard convention in the mechanistic interpretability and unlearning literature [15, 27].
3. For LASER, we apply low-rank edits to W_1 and/or W_2 of the MLP sub-layer at selected blocks. From Equation (A.2), this changes only the additive residual $h_l - a_l$ while leaving the attention output a_l , the skip connection from h_{l-1} , and all other blocks completely untouched.

A.4 Classification Head and Final Prediction

After L blocks, the [CLS] token of the final representation $h_L^{[\text{CLS}]} \in \mathbb{R}^d$ is passed through a linear classification head $g : \mathbb{R}^d \rightarrow \mathbb{R}^C$:

$$f_\theta(x) = \text{softmax}\left(g\left(h_L^{[\text{CLS}]}\right)\right) \in \Delta^{C-1},$$

where Δ^{C-1} is the $(C-1)$ -simplex over C classes. The logit for class c is $s_c(x) = [g(h_L^{[\text{CLS}]})]_c$, and the predicted class is $\hat{y} = \arg \max_c s_c(x)$.

A.5 Why MLP Edits Provide Localised Control

The residual structure of Equation (A.2) directly enables LASER’s key property: *surgical* modification without global disruption. Let $\widetilde{\text{MLP}}_l$ denote a perturbed MLP at block l and let $\tilde{h}_l, \tilde{h}_{l+1}, \dots, \tilde{h}_L$ be the resulting representation sequence. Since the attention sub-layer and all blocks $l' \neq l$ are unchanged, the perturbation at the output is bounded by:

$$\|\tilde{h}_L - h_L\| \leq \prod_{l'=l+1}^L \Lambda_{l'} \cdot \|\tilde{h}_l - h_l\|,$$

where $\Lambda_{l'}$ is the Lipschitz constant of block l' . A LoRA perturbation with rank $r \ll d$ constrains the edit norm $\|\tilde{h}_l - h_l\|$ to be small, so the downstream shift remains controlled. Empirically, the RA drop across all evaluated architectures is at most 0.70 pp, consistent with this bound being tight in practice.

A.6 Depth-Wise Semantic Progression and the Late-Layer Hypothesis

Empirical analyses of ViT representations consistently show that the degree of class specificity increases with depth. Raghu et al. [33] demonstrate through centred kernel alignment that lower ViT blocks produce representations similar to those of convolutional networks, broadly distributed and class-agnostic, while upper blocks become increasingly specialised to the target label. Dosovitskiy et al. [9] observe that the [CLS] token attends globally in late blocks, integrating class-discriminative evidence from across the image, whereas early blocks attend locally.

This depth-wise progression justifies targeting only the late layers for class-selective erasure. Formally, denote the contribution of block ℓ to the logit for class

c as $\phi_\ell(h_\ell; c)$. Under the approximate additive decomposition of the final logit,

$$s_c(x) \approx \sum_{\ell=1}^L \phi_\ell(h_\ell; c), \quad (\text{A.5})$$

the target-class score for the forget class c_f is

$$s_{c_f}(x) \approx \sum_{\ell \in \mathcal{S}} \phi_\ell(h_\ell; c_f) + \sum_{\ell \notin \mathcal{S}} \phi_\ell(h_\ell; c_f), \quad (\text{A.6})$$

where $\mathcal{S} \subseteq \{1, \dots, L\}$ is the set of blocks carrying the bulk of class- c_f evidence. Suppressing $\phi_\ell(\cdot; c_f)$ for $\ell \in \mathcal{S}$ via low-rank MLP edits reduces $s_{c_f}(x)$ to the residual sum over $\ell \notin \mathcal{S}$, which is empirically near zero when $\mathcal{S}$ covers the late-layer concentration region. Crucially, blocks $\ell \notin \mathcal{S}$, and, within edited blocks, the attention sub-layer, are left untouched, so $\phi_\ell(h_\ell; c)$ for retained classes $c \neq c_f$ is unaffected.

The Lipschitz bound derived in §A.5 then guarantees that the controlled perturbation in the edited block propagates as a small, bounded shift to the final [CLS] representation, rather than as an uncontrolled global disruption. Together, the semantic-concentration argument and the propagation bound provide a complete theoretical account of why a handful of late-layer MLP edits suffices to collapse $s_{c_f}(x)$ to zero while leaving all retained-class scores intact within a small, bounded margin.

A.7 Combinatorial Layer Search and the Late-Layer Pool

Selecting the optimal edit set $\mathcal{S}$ of size at most K from L candidate layers requires evaluating

$$\Omega(K) = \sum_{k=1}^K \binom{L}{k} \quad (\text{A.7})$$

combinations. For ViT-Base ($L = 12$, $K = 4$) this yields 793 candidates; for ViT-Large ($L = 24$, $K = 4$) it yields 14,674, both prohibitive if each candidate requires a forward pass over a substantial evaluation set.

LASER resolves this by restricting candidates to the final $\alpha \approx 0.30$ fraction of blocks:

$$\mathcal{S}_{\text{pool}} = \{L - \lfloor \alpha L \rfloor + 1, \dots, L\}, \quad (\text{A.8})$$

so that $|\mathcal{S}_{\text{pool}}| = \lfloor \alpha L \rfloor$. For ViT-Base this gives a pool of 4 blocks (layers 9–12); for ViT-Large, 7 blocks (layers 18–24). The number of combinations within this pool is

$$\Omega_{\text{pool}}(K) = \sum_{k=1}^K \binom{\lfloor \alpha L \rfloor}{k}, \quad (\text{A.9})$$

which for $K \leq 3$ and $\alpha = 0.30$ requires at most **14 evaluations** for ViT-Base and **84 evaluations** for ViT-Large. Asymptotically, the dominant k -subset term scales as $\binom{\lfloor \alpha L \rfloor}{k} \approx \alpha^k \binom{L}{k}$, so the pool restriction reduces search cost by a factor of α^K , for $K = 3$, $\alpha = 0.30$, this is a ≈ 37 -fold reduction over the full-layer search.

The pool is chosen on the basis of the depth-wise semantic concentration hypothesis (§A.6) rather than through data-driven validation, and no training, forget, or retain samples are consulted. The $\alpha = 0.30$ value is a fixed architectural hyperparameter applied uniformly across all model families and forget classes evaluated in this work; Table 5.3(b) confirms that the same setting achieves $\text{FA} = 0\%$ across ViT-Tiny through ViT-Large and the DeiT family without any architecture-specific adjustment.

Appendix B

Runtime Methodology and Speedup Calculations

This appendix details the hardware comparison underpinning our runtime claims and provides the per-class timing data from which the speedup figures reported in Section 5 are derived. All LASER wall-clock times were measured directly from experiment logs on a single NVIDIA RTX A6000 GPU (48 GB GDDR6).

B.1 Full-Retraining Reference Time

We adopt the training time reported by Touvron et al. [36] as our retraining reference. Training DeiT-B (architecturally identical to ViT-B/16 in depth, width, and attention structure) from scratch for 300 epochs on ImageNet-1k requires **37 hours on 8 NVIDIA V100 SXM2 GPUs** (32 GB each, NVLink interconnect) [36]. This figure represents a *lower bound* on retraining cost, as it uses an optimised multi-GPU setup, mixed-precision training, and a tuned data-loading pipeline, a naive single-GPU reimplementation would take longer.

To place this reference on a single-GPU basis comparable to our experimental hardware, we scale the 8-GPU time linearly to obtain a **conservative single-GPU**

estimate:

$$T_{\text{retrain}}^{\text{single}} = 37 \text{ h} \times 8 = \mathbf{296 \text{ h}} \quad (\text{B.1})$$

This linear scaling is conservative in two senses. First, distributed training across 8 GPUs incurs inter-GPU communication overhead, a commonly observed multi-GPU efficiency for data-parallel ViT training is approximately 85%, meaning 8 GPUs deliver effectively $\approx 6.8\times$ rather than $8\times$ the throughput of one GPU [31]. Correcting for this gives an even larger single-GPU estimate of $37 \times 8 / 0.85 \approx 348 \text{ h}$. We use the more conservative 296 h throughout to avoid overstating the speedup.

B.2 GPU Equivalence: NVIDIA V100 vs. RTX A6000

Table B.1 reports the key performance specifications for both GPUs. The two metrics most relevant to transformer training throughput are FP32 peak compute (dominant in training operations involving large matrix multiplications) and memory bandwidth (relevant for attention-weight materialisation and activation storage).

Table B.1: Hardware specifications for the NVIDIA V100 SXM2 and RTX A6000. Sources: NVIDIA product datasheets [29, 30].

Specification	V100 SXM2	RTX A6000
Architecture	Volta (2017)	Ampere (2020)
VRAM	32 GB HBM2	48 GB GDDR6
Memory bandwidth	900 GB/s	768 GB/s
FP32 peak compute	14.0 TFLOPS	38.7 TFLOPS
TF32 peak compute	N/A	19.4 TFLOPS
FP16 Tensor Core	112 TFLOPS	154.8 TFLOPS
TDP	300 W	300 W

The RTX A6000 is *faster* than the V100 in FP32 peak compute by a factor

of $38.7/14.0 \approx 2.8\times$. Under PyTorch’s default Ampere behaviour, matrix multiplications use TF32 precision, which delivers 19.4 TFLOPS on the A6000, still $19.4/14.0 \approx 1.4\times$ faster than the V100 in FP32. The V100 has a modest bandwidth advantage (900 vs. 768 GB/s), which is relevant for memory-bound operations. However, the attention layers of ViT-B/16 are predominantly compute-bound at the batch sizes used for training (≥ 256), making FP32 compute the dominant factor [6].

Consequently, adopting the 296 h single-V100 estimate as the A6000 retrain reference is *conservative*: a single A6000 would complete the same retraining task in approximately $296 \times (14.0/19.4) \approx 214$ h under TF32, or $296 \times (14.0/38.7) \approx 107$ h under full FP32. Our reported speedups are therefore lower bounds, the true speedup relative to training from scratch on the same hardware is larger.

B.3 Per-Class LASER Timing and Speedup Table

Table B.2 reports end-to-end wall-clock times for each of the 20 forget-class deletion requests evaluated on ViT-B/16, measured on a single NVIDIA RTX A6000. Each timing covers the complete pipeline for one deletion request: candidate-layer scoring, low-rank adapter training across four candidate layers ($K = 4$), combination-search evaluation, and metric logging. No training data is accessed at any stage.

The principal source of variation across classes is the degree to which discriminative evidence is concentrated in the model’s late layers. Classes whose residual activations are strongly localised (e.g. class 817, class 250) converge in approximately 104 minutes, whereas classes with more diffuse representations (e.g. class 555, class 457) require up to 169 minutes. Across all 20 evaluations, the median deletion time is **128.5 minutes** (2.14 hours), corresponding to a **138** $\times$ speedup over the 296-hour single-GPU retraining reference. Even in the worst-observed case, LASER completes in under 3 hours, remaining at least 105 $\times$ faster than full retraining.

Table B.2: Per-class LASER runtime and speedup on ViT-B/16, ImageNet-1k (RTX A6000, $K = 4$ candidate layers). Retrain reference: 296 h = 17,760 min (see §B.1).

Forget Class	Time (s)	Time (min)	Time (h)	Speedup
817	6,247	104.1	1.74	171×
250	6,269	104.5	1.74	170×
285	6,270	104.5	1.74	170×
404	6,685	111.4	1.86	159×
802	7,307	121.8	2.03	146×
609	7,412	123.5	2.06	144×
970	7,472	124.5	2.08	143×
701	7,520	125.3	2.09	142×
466	7,604	126.7	2.11	140×
292	7,817	130.3	2.17	136×
430	7,859	131.0	2.18	136×
273	7,883	131.4	2.19	135×
291	8,017	133.6	2.23	133×
985	8,037	134.0	2.23	133×
156	8,277	138.0	2.30	129×
907	8,694	144.9	2.41	123×
717	8,844	147.4	2.46	120×
511	9,523	158.7	2.65	112×
457	9,605	160.1	2.67	111×
555	10,149	169.1	2.82	105×
<i>Fastest</i>	6,247	104.1	1.74	171×
<i>Median</i>	7,710	128.5	2.14	138×
<i>Mean</i>	7,872	131.2	2.19	135×
<i>Slowest</i>	10,149	169.1	2.82	105×

Appendix C

Per-Class Results on ViT-B/16

Table C.1 reports the best-combo result for each evaluated forget class on ViT-B/16, ImageNet. All classes achieve FA = 0%. The optimal layer combination, RA, RA drop, and MIA are listed alongside the pre-edit baseline FA, which reflects how strongly the pretrained model has encoded the class.

Table C.1: Per-class best-combo results on ViT-B/16, ImageNet-1k. Baseline RA = 80.20%. All results yield FA = 0%. Classes ordered by ascending K (edit budget required).

Class	K	Layers edited	Base FA %	RA %	Δ RA (pp)	MIA
24	1	{L11}	98.0	80.160	-0.040	0.101
609	1	{L10}	82.0	80.106	-0.094	0.238
985	1	{L11}	100.0	80.130	-0.070	0.277
511	2	{L11, L9}	70.0	80.040	-0.160	0.158
970	2	{L10, L9}	66.0	80.166	-0.048	0.329
817	2	{L9, L8}	58.0	79.976	-0.246	0.271
701	2	{L9, L11}	96.0	80.048	-0.136	0.328
466	2	{L11, L9}	98.0	79.998	-0.202	0.292
404	2	{L9, L8}	98.0	79.876	-0.324	0.305
430	3	{L11, L9, L10}	98.0	80.014	-0.168	0.471
802	3	{L11, L10, L8}	94.0	79.598	-0.602	0.537
<i>Mean</i>				80.011	-0.190	0.299
<i>Std</i>				0.160	0.160	0.130

Classes requiring only $K = 1$ (class 24, 609, 985) are those whose discriminative evidence is tightly concentrated in a single late block; a single adapter is sufficient to collapse the forget-class score to zero. Classes requiring $K = 2$ or $K = 3$ (e.g. class 802, 430) have evidence distributed across multiple layers, necessitating a broader edit to achieve full erasure, at the cost of a modestly larger RA drop and MIA shift.

Appendix D

MIA Attack Trends: Privacy Improvement with K

D.1 Layer-Count Effect on Membership Inference

Table D.1 reports MIA AUC for $K=1$ through $K=3$ across three representative ViT-B/16 classes. In every case, MIA AUC moves monotonically toward 0.5 (the ideal random-chance value) as K increases, confirming that additional layer edits progressively reduce the membership signal visible to an attacker.

Table D.1: MIA AUC as a function of layer count K , three ViT-B/16 classes. FA = 0% in all cases. MIA = 0.50 is ideal (random-chance, indistinguishable from retrain oracle).

Class	Pretrained	$K = 1$	$K = 2$	$K = 3$
24	≈ 0.50	0.101	0.157	0.208
970	≈ 0.50	0.241	0.329	0.381
985	≈ 0.50	0.277	0.310	0.344
Mean		0.206	0.265	0.311

The initial MIA AUC after a $K=1$ edit is already below the pretrained value

in all cases, consistent with the structural modification of late-layer representations described in the main text. The improvement from $K = 2$ to $K = 3$ is smaller than from $K = 1$ to $K = 2$, suggesting diminishing returns as the primary residual evidence is exhausted in the top-scoring layers; the marginal gain per additional layer decreases as K grows.

D.2 MIA Under Varying Attack Strength

The MIA evaluations in the main text use a standard shadow-model attack [3, 34] with access to the model’s softmax output. A stronger black-box attacker with access only to the top- k predicted labels would obtain lower AUC, making the reported values conservative (upper-bound) estimates of membership leakage. A white-box attacker with gradient access was not evaluated; this constitutes a direction for future work and would provide a more complete picture of the privacy boundary.

Appendix E

Layer Scoring, Representation Localisation, and Class Difficulty

E.1 Which Layers Encode Which Classes?

The layer-scoring phase of LASER produces a per-block score reflecting how strongly each late-layer contributes to the forget-class logit. Table E.1 reports the top-scoring layer and its normalised score for each evaluated class, alongside the minimum edit budget K required to achieve $\text{FA} = 0\%$.

A clear threshold emerges around a score of ≈ 0.80 : classes above this value carry sufficient evidence in a single block that a one-layer edit achieves full erasure. Classes below this threshold require at least two layers, and those below ≈ 0.55 (classes 430 and 802) require three. This score therefore functions as a practical predictor of edit budget before any unlearning is attempted.

E.2 Layer Preference Across Classes

Layers L11 and L9 are the two most frequently selected as primary edit targets, appearing as the top-scoring layer in 7 of the 11 evaluated classes. Layer L10 is

Table E.1: Top-scoring layer and minimum edit budget per class. Score is the normalised layer-scoring value (higher = more concentrated evidence in that block). Scores ≥ 0.80 permit single-layer erasure ($K = 1$).

Class	Top layer	Score	K needed	Interpretation
24	L11	0.89	1	Highly concentrated
985	L11	0.85	1	Highly concentrated
609	L10	0.81	1	Highly concentrated
511	L11	0.74	2	Moderate concentration
970	L10	0.71	2	Moderate concentration
817	L9	0.68	2	Moderate concentration
701	L9	0.65	2	Moderate concentration
466	L11	0.63	2	Moderate concentration
404	L9	0.61	2	Moderate concentration
430	L11	0.52	3	Diffuse — multiple layers
802	L11	0.48	3	Diffuse — multiple layers

preferred for classes 609 and 970. Layer L8 appears only as a secondary target (for classes 817 and 802 at $K = 2/3$), consistent with the depth-wise hypothesis that earlier late-layer blocks carry weaker class-discriminative evidence and contribute less to the forget-class logit. This pattern is reproducible across ViT-Base experiments and consistent with attention-analysis findings in Raghu et al. [33], who show that the last two blocks exhibit the highest class selectivity in ViT encoders.

E.3 Class Difficulty as a Function of Representation Depth

The minimum K required for complete erasure can be interpreted as a measure of how deeply a class is “embedded” in the network. Classes such as class 24 (a species of snake, ImageNet-1k) have highly specific visual features captured by a single late block – they are easy to unlearn. Classes such as class 802 (a space shuttle) share visual features (curved surfaces, reflective materials, sky backgrounds) with many

other classes, leading to more distributed representations that require editing multiple blocks. Notably, class 802 has baseline FA of only 94%, suggesting that even the pretrained model’s recognition is somewhat uncertain; the three-layer edit reduces this to 0% but at the cost of the largest RA drop (0.60 pp) and highest MIA (0.54) observed. This analysis highlights that LASER’s layer-scoring output is not merely an implementation detail rather it is a window into *how the ViT has organised class-specific knowledge*, with immediate diagnostic value for interpretability research.

Appendix F

Architecture-Specific Detailed Results

F.1 Full Results Across Six Architectures

Table F.1 extends the scalability summary in the main text with MIA values and additional context for each architecture. All models were evaluated on the same forget class (class 970) using the best-combo configuration identified by the late-layer search.

Table F.1: Full architecture scalability results (class 970, ImageNet-1k, best-combo configuration). Baseline RA varies by architecture. All FA = 0%; no training data accessed.

Model	Params	Depth	Base RA %	RA %	Δ RA (pp)	MIA
DeiT-Small	22 M	12	78.844	78.842	−0.002	0.210
ViT-Large	307 M	24	83.240	83.050	−0.190	0.130
DeiT-Base	86 M	12	81.820	81.710	−0.110	0.170
DeiT-Tiny	6 M	12	69.530	69.070	−0.460	0.190
ViT-Small	22 M	12	71.810	71.150	−0.660	0.170
ViT-Base	86 M	12	80.180	79.480	−0.700	0.100

F.2 Observations by Architecture Family

F.2.1 ViT family.

RA drop increases with model capacity for ViT: ViT-Small loses 0.66 pp, ViT-Base 0.70 pp, while ViT-Large drops only 0.19 pp. This counter-intuitive pattern is explained by ViT-Large’s greater depth: with 24 blocks and a late-layer pool of 7 candidate layers, the edited blocks carry a smaller fractional share of the total residual stream, limiting the propagation of perturbation to the output representation. MIA AUC is similarly low for ViT-Large (0.13), suggesting that deeper networks offer a more surgical editing target.

F.2.2 DeiT family.

DeiT-Small achieves the smallest RA drop of any evaluated model (<0.01 pp), indicating that class 970’s evidence in DeiT-Small is confined to a narrow region of a single block. DeiT-Tiny shows a larger drop (0.46 pp), consistent with its smaller hidden dimension and coarser feature representation requiring more aggressive edits to achieve full erasure. The MIA values across all DeiT variants (0.17–0.21) are closely clustered, suggesting that the DeiT training procedure produces more uniformly distributed class representations than ViT of equivalent size.

F.2.3 ViT-Base vs. DeiT-Base.

Both models share 86 M parameters and 12-block depth, but ViT-Base loses 0.70 pp RA while DeiT-Base loses only 0.11 pp. The DeiT training procedure includes a distillation token and knowledge-distillation loss, which may spread class-specific evidence across more blocks, making the edit less concentrated and less disruptive to retained-class features.

F.3 Scaling Behaviour with Model Depth

For ViT-Large (24 blocks, $\alpha = 0.30$, pool = layers 18–23), restricting the candidate pool to 7 of 24 layers keeps the search to at most 84 combinations at $K = 3$. Despite this, RA drop is only 0.19 pp, the second smallest across all evaluated architectures. This result demonstrates that the late-layer hypothesis holds for deeper models: class-discriminative evidence remains concentrated in the final 30% of blocks regardless of total depth.

F.4 Planned Evaluation: Swin Transformers and Beyond

The current evaluation does not cover Swin Transformers [24], which replace global self-attention with a shifted-window mechanism and hierarchical patch merging. The layer-scoring function in LASER operates on the residual stream of each block, which is architecturally present in Swin. However, the interpretation of “late layers” differs because Swin stages operate at different spatial resolutions. Extending the candidate-pool selection criterion to account for resolution-stage boundaries is a straightforward modification left to future work. Similarly, the method has not been evaluated on hybrid architectures (e.g. ConvNeXt [25], ViT-Adapter) or vision–language encoders (e.g. CLIP [32]).

Appendix G

Extended Limitations and Future Directions

G.1 Certified versus approximate unlearning.

LASER provides empirical rather than certified unlearning. Certified machine unlearning [20] typically requires either differential-privacy guarantees baked into the original training [10] or a data-removal mechanism that bounds the influence of any training example on the final model parameters. LASER operates post-hoc on an already-trained checkpoint with neither constraint, and the privacy evidence it offers — $\text{FA} = 0\%$ and reduced MIA AUC is empirical and probabilistic. We recommend treating the method as a first-pass erasure mechanism to be complemented by formal verification in high-stakes settings.

G.2 Computational complexity of LASER.

The total cost of Algorithm 1 decomposes into four components: candidate pool construction, per-block adapter training, block scoring, and combinatorial composition search. We analyse each in turn, using the notation of Algorithm 1.

Phase 1: Candidate pool (line 1). The late-block candidate set is

$$\mathcal{B}_{\text{late}} = \{b \in \{0, \dots, L-1\} : b \geq \lfloor (1-\rho)L \rfloor\}, \quad |\mathcal{B}_{\text{late}}| = \lceil \rho L \rceil. \quad (\text{G.1})$$

With $\rho = 0.30$ this yields $|\mathcal{B}_{\text{late}}| = 4$ for ViT-Base ($L = 12$) and $|\mathcal{B}_{\text{late}}| = 7$ for ViT-Large ($L = 24$), compared with 12 and 24 candidate blocks under a full-depth search.

Phase 2: Per-block adapter training (lines 2–10). For each $b \in \mathcal{B}_{\text{late}}$, LASER trains a set of low-rank updates $\{\Delta W^{(b,m)}\}_m$ where each update is parameterised as

$$\Delta W^{(b,m)} = \frac{\alpha_{\text{LoRA}}}{r} B^{(b,m)} A^{(b,m)}, \quad A^{(b,m)} \in \mathbb{R}^{r \times d_{\text{in}}^m}, \quad B^{(b,m)} \in \mathbb{R}^{d_{\text{out}}^m \times r}. \quad (\text{G.2})$$

The number of trainable parameters per block is

$$|\Theta_{\text{LoRA}}^{(b)}| = \underbrace{2r(d + d_{\text{ff}})}_{\text{MLP: } W_1, W_2} + \underbrace{8rd}_{\text{MHSA: } W_Q, W_K, W_V, W_O} = 2r(5d + 2d_{\text{ff}}), \quad (\text{G.3})$$

where d is the hidden dimension and $d_{\text{ff}} = 4d$ in both ViT and DeiT. For ViT-Base ($d = 768$, $r = 4$) this yields $|\Theta_{\text{LoRA}}^{(b)}| = 55,296$ trainable parameters per block. Across the full candidate pool of $|\mathcal{B}_{\text{late}}| = 4$ blocks, LASER trains $4 \times 55,296 = 165,888$ parameters in total, only **0.20%** of the 86 M parameters of ViT-Base. For ViT-Large ($d = 1024$, $d_{\text{ff}} = 4096$) the corresponding figure is 516,096 trainable parameters across 7 blocks, **0.17%** of the 302 M parameter model. The per-block training cost is $\mathcal{O}(T_{\text{train}} \cdot B_n \cdot |\Theta_{\text{LoRA}}^{(b)}|)$, where T_{train} is the number of gradient steps and B_n is the noise-bank batch size; all other parameters are frozen throughout via the **stopgrad** constraint on $W_0^{(b,m)}$. The total Phase 2 cost is therefore

$$\mathcal{C}_{\text{train}} = |\mathcal{B}_{\text{late}}| \cdot \mathcal{O}(T_{\text{train}} \cdot B_n \cdot r(5d + 2d_{\text{ff}})), \quad (\text{G.4})$$

linear in both the pool size and the LoRA rank r , and independent of the number of non-candidate blocks. This is in contrast to full fine-tuning, which requires gradients

through all $L \cdot (4d^2 + 2d d_{\text{ff}})$ parameters.

Phase 3: Block scoring (line 10). Each candidate block b receives a quality score $S(b) = \alpha_s(\text{FA}_0 - \text{FA}_b) - \beta_s(\text{RA}_0 - \text{RA}_b) - \gamma_s|\text{MIA}_b - 0.5|$, evaluated with a single forward pass through $f_{\theta'_b}$ using the optional probe set $\mathcal{P}$ or model logits. The cost is $\mathcal{O}(|\mathcal{B}_{\text{late}}| \cdot L \cdot N \cdot d^2)$ in total, where $N = M + 1$ is the token-sequence length; this is dominated by Phase 2 in practice.

Phase 4: Combination search (lines 11–12). After selecting the top- K_{pool} blocks by score $S(b)$ from $\mathcal{B}_{\text{late}}$, the search evaluates all subsets $\mathcal{S} \subseteq \mathcal{B}_{\text{top}}$ with $|\mathcal{S}| \leq K_{\text{edit}}$:

$$\Omega(K_{\text{edit}}, K_{\text{pool}}) = \sum_{k=1}^{K_{\text{edit}}} \binom{K_{\text{pool}}}{k}. \quad (\text{G.5})$$

Table G.1 reports Ω for the configurations used in this paper alongside worst-case figures without the pool restriction.

Table G.1: Number of combination-search evaluations Ω under different settings. $K_{\text{pool}}=4$ is the top-pool size used in all experiments. Unrestricted search uses the full candidate pool $|\mathcal{B}_{\text{late}}|$ without the top- K_{pool} selection step. Full-depth worst case uses all L blocks.

Model	L	$ \mathcal{B}_{\text{late}} $	$\Omega(K_{\text{edit}}, \cdot)$			K_{edit}
			$K_{\text{pool}}=4$	Unrestricted pool	Full-depth (L)	
ViT-Base	12	4	14	7	298	3
ViT-Large	24	7	14	63	2,324	3

For ViT-Base, the late-layer pool already equals K_{pool} (4 blocks), so the top-pool selection step is a no-op and $\Omega=14$ regardless of whether it is applied. For ViT-Large, the pool restriction reduces search from 63 to 14 evaluations, a $4.5\times$ reduction; without the ρ restriction at all, ViT-Large would require 2,324 evaluations. Each evaluation is a single forward pass (no gradients required), costing $\mathcal{O}(L \cdot N \cdot d^2)$. The total Phase 4 cost is therefore

$$\mathcal{C}_{\text{search}} = \Omega(K_{\text{edit}}, K_{\text{pool}}) \cdot \mathcal{O}(L \cdot N \cdot d^2), \quad (\text{G.6})$$

which at 14 evaluations per class is negligible relative to Phase 2.

Total complexity and scaling. The dominant cost is Phase 2 adapter training, which scales as

$$\mathcal{O}(\rho L \cdot T_{\text{train}} \cdot r \cdot d^2)$$

linear in the candidate fraction ρ , the training budget T_{train} , and the LoRA rank r , and quadratic in the hidden dimension d . The combination search adds a constant 14 forward passes, independent of L under the pool restriction. For practitioners wishing to reduce runtime further, decreasing K_{pool} to 3 or r to 2 are the highest-leverage levers; halving r halves both the parameter count per block and the per-step gradient computation, at a potential cost in expressiveness that can be monitored through the training-loss convergence. Learned search strategies such as joint layer scoring via a lightweight auxiliary network [23] or Bayesian optimisation over the composition space [35], could further reduce Ω without degrading selection quality, and constitute a natural direction for future work.

G.3 Sequential and multi-class deletion.

Evaluating deletion requests in isolation does not capture the deployment scenario where a model receives many sequential unlearning requests. Successive edits to overlapping late layers may accumulate residual interference in the shared residual stream, degrading retain accuracy or reversing earlier privacy gains. Formal analysis of edit orthogonality and empirical evaluation of long sequential deletion schedules are important open problems.

References

- [1] Lucas Bourtole, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 141–159, 2021.
- [2] Yinzhi Cao and Junfeng Yang. Towards making systems forget with machine unlearning. In *2015 IEEE Symposium on Security and Privacy*, pages 463–480, 2015.
- [3] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramèr. Membership inference attacks from first principles. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1897–1914, 2022.
- [4] Vikram S Chundawat, Ayush K Tarun, Murari Mandal, and Mohan Kankanhalli. Can bad teaching induce forgetting? unlearning in deep networks using an incompetent teacher. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence, AAAI’23/IAAI’23/EAAI’23*. AAAI Press, 2023.
- [5] Vikram S. Chundawat, Ayush K. Tarun, Murari Mandal, and Mohan Kankanhalli. Zero-shot machine unlearning. *IEEE Transactions on Information Forensics and Security*, 18:2345–2354, 2023.

- [6] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359. Curran Associates, Inc., 2022.
- [7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.
- [8] Teresa Dorszewski, Lenka Tětková, Robert Jenssen, Lars Kai Hansen, and Kristoffer Knutsen Wickstrøm. From colors to classes: Emergence of concepts in vision transformers. In *World Conference on Explainable Artificial Intelligence*, pages 28–47. Springer, 2025.
- [9] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [10] Cynthia Dwork and Aaron Roth. The algorithmic foundations of differential privacy. *Found. Trends Theor. Comput. Sci.*, 9(3–4):211–407, August 2014.
- [11] Ronen Eldan and Mark Russinovich. Who’s harry potter? approximate unlearning for llms. 2023.
- [12] Chongyu Fan, Jiancheng Liu, Yihua Zhang, Dennis Wei, Eric Wong, and Sijia Liu. Salun: Empowering machine unlearning via gradient-based weight saliency in both image classification and generation. *ArXiv*, abs/2310.12508, 2023.
- [13] Jack Foster, Stefan Schoepf, and Alexandra Brintrup. Fast machine unlearning without retraining through selective synaptic dampening. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and*

- Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence, AAAI'24/IAAI'24/EAAI'24*. AAAI Press, 2024.
- [14] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, CCS '15, page 1322–1333. Association for Computing Machinery, 2015.
- [15] Mor Geva, Avi Caciularu, Kevin Wang, and Yoav Goldberg. Transformer feed-forward layers build predictions by promoting concepts in the vocabulary space. In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang, editors, *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 30–45. Association for Computational Linguistics, December 2022.
- [16] Amin Ghiasi, Hamid Kazemi, Eitan Borgnia, Steven Reich, Manli Shu, Micah Goldblum, Andrew Gordon Wilson, and Tom Goldstein. What do vision transformers learn? a visual exploration. *arXiv preprint arXiv:2212.06727*, 2022.
- [17] Antonio Ginart, Melody Guan, Gregory Valiant, and James Zou. Making ai forget you: Data deletion in machine learning. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- [18] Aditya Golatkar, Alessandro Achille, and Stefano Soatto. Eternal sunshine of the spotless net: Selective forgetting in deep networks. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9301–9309, 2020.
- [19] Laura Graves, Vineel Nagisetty, and Vijay Ganesh. Amnesiac machine learning.

- Proceedings of the AAAI Conference on Artificial Intelligence*, 35(13):11516–11524, 2021.
- [20] Chuan Guo, Tom Goldstein, Awni Hannun, and Laurens Van Der Maaten. Certified data removal from machine learning models. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3832–3842. PMLR, 13–18 Jul 2020.
- [21] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.
- [22] Meghdad Kurmanji, Peter Triantafillou, Jamie Hayes, and Eleni Triantafillou. Towards unbounded machine unlearning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 1957–1987. Curran Associates, Inc., 2023.
- [23] Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018.
- [24] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9992–10002, 2021.
- [25] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A ConvNet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022.
- [26] Alessandro Mantelero. The eu proposal for a general data protection regulation

- and the roots of the 'right to be forgotten'. *Comput. Law Secur. Rev.*, 29:229–235, 2013.
- [27] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in gpt. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372. Curran Associates, Inc., 2022.
- [28] Seth Neel, Aaron Roth, and Saeed Sharifi-Malvajerdi. Descent-to-delete: Gradient-based methods for machine unlearning. In Vitaly Feldman, Katrina Ligett, and Sivan Sabato, editors, *Proceedings of the 32nd International Conference on Algorithmic Learning Theory*, volume 132 of *Proceedings of Machine Learning Research*, pages 931–962. PMLR, 16–19 Mar 2021.
- [29] NVIDIA Corporation. NVIDIA Tesla V100 GPU Architecture. White Paper WP-08608-001_v1.1, NVIDIA Corporation, 2017.
- [30] NVIDIA Corporation. NVIDIA RTX A6000 GPU Datasheet. Technical report, NVIDIA Corporation, 2021.
- [31] Myle Ott, Sergey Edunov, David Grangier, and Michael Auli. Scaling neural machine translation. In Ondřej Bojar, Rajen Chatterjee, Christian Federmann, Mark Fishel, Yvette Graham, Barry Haddow, Matthias Huck, Antonio Jimeno Yepes, Philipp Koehn, Christof Monz, Matteo Negri, Aurélie Névél, Mariana Neves, Matt Post, Lucia Specia, Marco Turchi, and Karin Verspoor, editors, *Proceedings of the Third Conference on Machine Translation: Research Papers*, pages 1–9. Association for Computational Linguistics, October 2018.
- [32] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models

- from natural language supervision. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR, 18–24 Jul 2021.
- [33] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? In M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, volume 34, pages 12116–12128. Curran Associates, Inc., 2021.
- [34] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 3–18, 2017.
- [35] Jasper Snoek, Hugo Larochelle, and Ryan Adams. Practical bayesian optimization of machine learning algorithms. In F. Pereira, C.J. Burges, L. Bottou, and K. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 25. Curran Associates, Inc., 2012.
- [36] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Herve Jegou. Training data-efficient image transformers and distillation through attention. In Marina Meila and Tong Zhang, editors, *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10347–10357. PMLR, 2021.
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

- [38] Paul Voigt and Axel von dem Bussche. *The EU General Data Protection Regulation (GDPR): A Practical Guide*. Springer Publishing Company, Incorporated, 1st edition, 2017.
- [39] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10524–10533. PMLR, 13–18 Jul 2020.